

BFT-SMART: Uma Ferramenta Robusta para Replicação Máquina de Estados

Alysson Neves Bessani¹, João Sousa¹ e Eduardo Adilio Pelinson Alchieri²

¹LaSIGE, Universidade de Lisboa, Lisboa - Portugal

²CIC, Universidade de Brasília, Brasília - Brasil

Abstract. *The last fifteen years have seen an impressive amount of work on protocols for Byzantine fault-tolerant (BFT) state machine replication (SMR). However, there is still a lack of practical and reliable software libraries implementing this technique. BFT-SMART is an open-source Java-based library implementing robust BFT state machine replication. When compared to other SMR libraries, BFT-SMART achieves better performance.*

Resumo. *Nos últimos anos surgiram muitos protocolos para replicação Máquina de Estados (SMR) tolerante a falhas bizantinas (BFT). No entanto, ainda faltava uma implementação confiável desta técnica. Visando suprir esta deficiência, o BFT-SMART é uma biblioteca Java de código aberto que implementa uma ferramenta robusta para BFT SMR. Além disso, o BFT-SMART apresenta um desempenho melhor do que outras ferramentas similares.*

URL do sistema (download, manuais, documentação e aplicação):

<http://code.google.com/p/bft-smart/>

1. Introdução

Nos últimos anos surgiram muitos protocolos para replicação Máquina de Estados (SMR) tolerante a falhas bizantinas (BFT) (ex., [Castro and Liskov 2002, Guerraoui et al. 2010, Veronese et al. 2013]), no entanto poucas implementações completas desta técnica têm sido desenvolvidas. O PBFT [Castro and Liskov 2002] e o UpRight [Clement et al. 2009] são gratas exceções, que infelizmente não são mais mantidas. Desta forma, as organizações que necessitam deste serviço precisam desenvolver sua própria implementação (ex., [Chandra et al. 2007]).

Neste artigo descrevemos o BFT-SMART, uma implementação Java robusta para BFT SMR que utiliza um protocolo similar ao PBFT. O BFT-SMART foi desenvolvido buscando-se um alto desempenho em execuções livres de falhas e corretude nos casos em que replicas faltosas apresentem um comportamento arbitrário. Além disso, o BFT-SMART é a primeira implementação de BFT SMR que permite reconfigurações no conjunto de replicas [Lamport et al. 2010] e fornece um suporte eficiente e transparente para serviços duráveis [Bessani et al. 2013a].

A principal contribuição deste trabalho é a documentação da implementação e da forma de utilização de um sistema para BFT SMR, preenchendo esta lacuna encontrada na literatura. Adicionalmente, apresentamos uma avaliação experimental do BFT-SMART, a qual compara este sistema com outras propostas similares e discute alguns *tradeoffs* relacionados com a tolerância a falhas por parada (*crash*) vs. falhas bizantinas.

2. Projeto do BFT-SMART

O desenvolvimento do BFT-SMART iniciou-se no ano de 2007 mas somente em 2011 tivemos uma implementação completa e robusta. Além de beneficiar-se de arquiteturas *multi-cores*, não utilizar otimizações complexas e possuir uma API simples (Seção 5), o projeto do BFT-SMART segue os seguintes princípios [Bessani et al. 2013b]:

Modelo de falhas configurável. Por padrão, o BFT-SMART tolera (1) atraso, perda e corrupção de mensagens e (2) comportamento arbitrário de processos. No entanto, este sistema ainda fornece assinaturas criptográficas (para tolerar falhas bizantinas) e um protocolo de SMR simplificado (para tolerar apenas *crashes*). A menos que seja explicitado no texto, focamos nossa discussão na configuração para falhas bizantinas.

Modularidade. O BFT-SMART implementa um protocolo de SMR modular, que utiliza uma primitiva de consenso bem definida [Sousa and Bessani 2012]. Além disso, existem módulos para comunicação ponto a ponto confiável, ordenação de requisições, transferência de estado e reconfiguração. Esta abordagem facilita a manutenção e o aprimoramento do sistema. Outros sistemas, como o PBFT, implementam um protocolo monolítico onde não existe uma separação entre os algoritmos.

2.1. Modelo de Sistema

O BFT-SMART assume o modelo de sistema usual para BFT SMR [Castro and Liskov 2002, Guerraoui et al. 2010]: $n = 3f + 1$ replicas para tolerar até f falhas Bizantinas; um número ilimitado de clientes sujeitos a falhas Bizantinas e sincronia parcial para garantir terminação. Como o sistema permite reconfigurações, n e f podem ser modificados durante a execução (Seção 2.2). Além disso, o sistema também pode ser configurado para usar somente $n = 2f + 1$ replicas e tolerar até f faltas por *crash* (Seção 4).

Independente da configuração, o sistema necessita de canais ponto a ponto confiáveis, os quais são implementados usando MACs sobre TCP/IP. As chaves simétricas necessárias nestes canais para comunicação entre as replicas são geradas usando o protocolo *Signed Diffie-Helman*, usando um par de chaves RSA para cada replica. Já as chaves necessárias para comunicação entre clientes e replicas são geradas baseadas nos seus identificadores, não necessitando um par de chaves por cliente. Também é possível configurar o sistema para garantir a autenticidade das requisições dos clientes, neste caso assinaturas são habilitadas e cada cliente precisará de um par de chaves (Seção 4).

2.2. Módulos Principais

Esta seção discute brevemente os principais protocolos implementados no BFT-SMART.

Multicast com Ordem Total. *Multicast* com ordem total é implementado através da utilização de um protocolo de consenso subjacente [Sousa and Bessani 2012]. Em uma execução normal, os clientes enviam suas requisições para todas as replicas e aguardam as respostas. A ordem total é conseguida através de uma sequência de instâncias de consenso, cada uma decidindo a ordem de entrega de um lote de requisições. Cada instância necessita de três passos de comunicação para terminar. Primeiro a replica líder do consenso envia uma proposta (*PROPOSE*) para as outras replicas, seguindo-se mais dois passos de comunicação (*WRITE* e *ACCEPT*) onde cada replica envia uma mensagem para todas as outras replicas. As mensagens *PROPOSE* contêm um lote de requisições enquanto que *WRITE* e *ACCEPT* contêm apenas um *hash* criptográfico destes lotes.

A descrição acima refere-se ao comportamento deste módulo em uma execução normal, que ocorre quando não ocorrem falhas e na presença de sincronia. Quando estas condições não são satisfeitas, este módulo chaveia para uma *fase de sincronização* onde um novo líder é escolhido e todas as replicas “saltam” para a mesma instância do consenso. Este salto pode fazer com que alguma replica inicie o protocolo de transferência de estado, abaixo descrito.

Transferência de Estado. Uma implementação de SMR completa (que possa ser usada na prática) deve possibilitar a recuperação e reintegração de replicas, sem ser necessário reiniciar o sistema todo. Além disso, memória estável deve ser utilizada para recuperar o sistema como um todo, caso possíveis falhas correlatas comprometam mais do que f replicas. O BFT-SMART implementa técnicas eficientes de durabilidade, como descrito em [Bessani et al. 2013a], para recuperar replicas ou todo o sistema. As idéias principais destas técnicas são: (1) armazenar os lotes de requisições em um disco enquanto estas requisições são executadas; (2) obter *snapshots* em diferentes pontos da execução em diferentes replicas para não precisar parar o sistema; e (3) transferir o estado de forma colaborativa, com cada replica enviando diferentes partes do estado para a replica sendo recuperada. Todas estas técnicas são implementadas em uma camada bem definida entre o protocolo de replicação e a aplicação, sem influenciar no protocolo de consenso.

Reconfiguração. Diferente de outros sistemas, o BFT-SMART fornece um protocolo que possibilita a adição e remoção de replicas durante a execução do sistema. Para isso, utiliza um tipo especial de cliente, chamado *View Manager*, que deve ser confiável e usado por administradores do sistema. O protocolo de reconfiguração funciona da seguinte forma: o *View Manager* envia uma requisição especial de reconfiguração para o sistema, informando a replica que deve ser adicionada ou removida do sistema. Como estas requisições são ordenadas (como qualquer outra requisição), todas as replicas corretas adotam a mesma visão (configuração) atual do sistema em qualquer ponto da execução das requisições de clientes. As requisições de reconfiguração são assinadas para garantir que o emissor é o *View Manager* (administrador) e não um cliente qualquer do sistema.

Após a requisição de reconfiguração ser ordenada, a mesma não é entregue para a aplicação como acontece com as requisições normais. Ao invés disso, as replicas atualizam suas visões atuais do sistema de acordo com as solicitações contidas na requisição e enviam uma resposta para o *View Manager* informando sobre o sucesso no processamento das atualizações. Caso positivo, o *View Manager* envia uma mensagem especial para a(s) replica(s) que está(ão) aguardando para entrar (sair) no sistema, informando que as mesmas já podem iniciar (deixar) a execução. Finalmente, as replicas adicionadas no sistema iniciam o protocolo de transferência de estado para suas atualizações.

3. Arquitetura e Implementação do BFT-SMART

O BFT-SMART contém menos do que 13.5K linhas de código Java divididos em pouco mais de 90 arquivos. Outros sistemas possuem um código muito maior: o PBFT [Castro and Liskov 2002] contém 20K linhas de código C, o UpRight [Clement et al. 2009] contém 22K linhas de código Java e o JPaxos [Santos and Schiper 2013] possui mais do que 22k linhas de código Java.

O principal desafio no projeto e implementação de um sistema de replicação com alto desempenho está na organização das várias tarefas dos protocolos em uma arquitetura robusta e eficiente. No caso de sistemas para BFT SMR, temos dois outros requisitos: o

sistema deve suportar centenas de clientes e resistir a comportamentos maliciosos tanto de replicas quanto de clientes. A Figura 1 apresenta a arquitetura do BFT-SMART com as *threads* utilizadas no processamento das requisições. Em nossa arquitetura, todas as *threads* comunicam-se através de filas (*bounded queues*). A figura mostra quais *threads* produzem/consomem dados em cada fila.

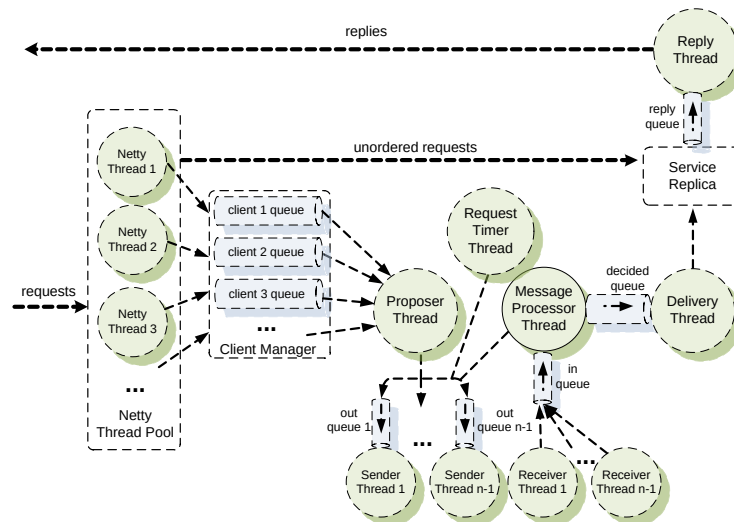


Figura 1. Processamento de requisições em uma replica do BFT-SMART.

As requisições de clientes são recebidas através de um *pool* de *threads* implementadas pelo *framework* de comunicação *Netty*, fazendo com que o BFT-SMART forneça um suporte eficiente para centenas de conexões entre clientes e replicas. Quando uma requisição é recebida, primeiramente é verificado se a mesma precisa ser ordenada (o BFT-SMART suporta requisições, geralmente de apenas leitura, que não precisam de ordenação). Requisições não ordenadas são diretamente entregues para a aplicação implementada (replica). As outras requisições são encaminhadas ao *client manager* que verifica a integridade das mesmas e as adiciona nas respectivas filas de cada cliente.

A *thread proposer* é responsável por criar os lotes de requisições e enviar a mensagem de proposta (*PROPOSE*) do protocolo de consenso. O BFT-SMART adiciona requisições em um lote até que (1) o número de requisições no lote atinja um limite especificado em um arquivo de configuração ou (2) não existam mais requisições pendentes. Esta *thread* apenas é ativada na replica líder do consenso.

Toda mensagem m que será enviada para outra replica é adicionada em uma fila de saída (*out queue*). Após isso, uma *thread* de envio (*sender thread*) remove m desta fila, serializa m , produz o MAC a ser adicionado na mensagem e a envia usando *sockets* TCP. Na replica receptora, uma *thread* de recebimento (*receiver thread*) para o dado emissor recebe m , autentica m (valida seu MAC), deserializa m e adiciona esta mensagem em uma fila de entrada (*in queue*) que armazena as mensagens recebidas de todas as replicas que ainda não foram processadas.

A *thread* de processamento de mensagens (*message processor thread*) é responsável pelo processamento das mensagens do protocolo BFT SMR. Esta *thread* recupera mensagens da fila de entrada e as processa caso sejam referentes à instância do consenso sendo executada. As mensagens relacionadas com uma instância seguinte do consenso são processadas apenas quando tal instância é inicializada.

Quando uma instância do consenso termina em uma replica, o lote decidido é adicionado na fila de decididos (*decided queue*). Uma *thread* de entrega (*delivery thread*) é responsável por retirar os lotes desta fila, deserializar todas as requisições do lote, remover as respectivas requisições das filas dos respectivos clientes e marcar a instância corrente do consenso como finalizada. Após isso, esta *thread* executa a aplicação implementada (*service replica*), executando as requisições e gerando as respectivas respostas que são adicionadas em uma fila de respostas (*reply queue*). Por fim, uma *thread* de respostas (*reply thread*) remove as respostas desta fila e as envia para os respectivos clientes.

Outra *thread* que merece destaque é a *request timer thread*, que é ativada periodicamente para verificar se alguma requisição permaneceu mais do que um tempo pré-definido na fila de requisições pendentes (de clientes). O primeiro *timeout* para uma requisição faz com que a mesma seja encaminhada para o líder corrente. Já o segundo *timeout* para a mesma requisição faz com que a instância corrente do consenso seja parada e a fase de sincronização seja executada (Seção 2.2). A ideia por traz destes *timeouts* é a seguinte: em condições normais da rede, um *timeout* é causado por um cliente que não enviou a requisição para o líder ou pelo líder que não ordenou a requisição de um cliente. Desta forma, primeiramente assumimos que o problema está no cliente e o líder é suspeito apenas se o problema persistir.

4. Configurações Alternativas

Como mencionado, o BFT-SMART pode ser configurado para tolerar tanto falhas por *crash* quanto falhas maliciosas.

Tolerância a Falhas por Crash (CFT). Um parâmetro de configuração é utilizado para indicar que o sistema deve ser CFT. Nesta configuração, o sistema tolera $f < n/2$ falhas (minorias simples), implicando em algumas mudanças nos protocolos (ex.: as mensagens *WRITE* do protocolo de consenso não são mais necessárias).

Tolerância a Falhas Bizantinas. O BFT-SMART pode ser configurado para utilizar criptografia de chave pública, fazendo cada cliente assinar suas requisições. Isso evita que clientes realizem ataques ao sistema para forçar a troca de líder [Amir et al. 2011].

5. API e Modelo de Programação

Para implementar uma aplicação baseada no BFT-SMART, duas classes principais são utilizadas. A classe *ServiceReplica* é utilizada no lado do servidor para instanciar uma replica da aplicação enquanto que a classe *ServiceProxy* é utilizada no lado do cliente para acessar a aplicação replicada. Para instanciar uma *ServiceReplica*, é necessário fornecer um identificador numérico (que é mapeado para um IP e uma porta através de um arquivo de configuração) e implementar as interfaces *Executable* (que define os métodos acessados pelo BFT-SMART quando a aplicação deve executar uma requisição) e *Recoverable* (que define os métodos acessados pelo BFT-SMART para gerenciamento do estado da aplicação). Usualmente, estas duas interfaces são implementadas por uma única classe (veja abaixo). No lado do cliente, para instanciar um *ServiceProxy* apenas é necessário o identificador numérico do cliente. O restante desta seção discute brevemente a API do BFT-SMART, que é descrita em detalhes na página da ferramenta [BFT-SMART, Bessani et al. 2013b].

Lado do Servidor. A forma mais simples de se implementar uma aplicação replicada usando o BFT-SMART é através da classe abstrata *DefaultSingleRecoverable*,

que implementa as interfaces `Executable` e `Recoverable` acima descritas. Para isso, o desenvolvedor precisa estender esta classe abstrata e implementar os seguintes métodos abstratos:

```
public byte[] executeOrdered(byte[] cmd, MsgContext ctx);
public byte[] executeUnordered(byte[] cmd, MsgContext ctx);
public byte[] getSnapshot();
public void installSnapshot(byte[] state);
```

O BFT-SMART invoca os métodos `executeOrdered` e `executeUnordered` para entregar as requisições (comandos) dos clientes para a aplicação. O primeiro método é executado quando o cliente solicita requisições que devem ser ordenadas, enquanto que o segundo é utilizado para requisições que não são ordenadas (usualmente requisições que apenas fazem leituras no estado da aplicação). Estes métodos devem implementar a parte principal da aplicação (o código da aplicação deve estar dentro destes métodos) e retornar respostas que serão enviadas aos clientes pelo BFT-SMART. O argumento `cmd` representa a requisição do cliente serializada e `ctx` contém meta-dados como o identificador do cliente, o identificador da instância do consenso que ordenou a requisição, a latência do consenso, etc. Além disso, para o gerenciamento do estado da aplicação, os desenvolvedores devem implementar os métodos `getSnapshot` e `installSnapshot` para criar e instalar *snapshots* serializados do estado da aplicação, respectivamente.

A classe `DefaultSingleRecoverable` geralmente é suficiente para implementação de aplicações baseadas no BFT-SMART. No entanto, uma aplicação pode utilizar implementações customizadas das interfaces `Executable` e `Recoverable` (veja [Bessani et al. 2013b] para mais detalhes).

Lado do Cliente. No lado do cliente, cada instância da classe `ServiceProxy` representa um cliente diferente com um identificador distinto. Esta classe fornece os seguintes métodos para enviar requisições (comandos) para a aplicação:

```
public byte[] invokeOrdered(byte[] request);
public byte[] invokeUnordered(byte[] request);
public void invokeAsynchronous(byte[] request,
    ReplyListener listener, int[] receivers, MsgType type);
```

Em todos estes métodos, requisições e respostas devem ser serializadas em um *array* de *bytes*, permitindo que os mesmos sejam genéricos e suportem qualquer aplicação. Os métodos `invokeOrdered` e `invokeUnordered` são utilizados para enviar requisições que serão e não serão ordenadas, respectivamente. O método `invokeAsynchronous` pode ser utilizado para envio de requisições (tanto ordenadas quanto não ordenadas – definido em `MsgType`) de forma não bloqueante, i.e., este método retorna sem aguardar pelas respostas das replicas. Através deste método programadores podem criar aplicações que continuam executando enquanto que o BFT-SMART coleta as respostas em *background*. Para utilizar este artifício, os programadores devem fornecer uma classe para *callback* definida pela interface `ReplyListener`, a qual deve gerenciar o recebimento das respostas.

A classe `ServiceProxy` geralmente é suficiente para implementação de aplicações baseadas no BFT-SMART. No entanto, clientes também podem sofrer customizações como por exemplo na forma de contar as respostas [Bessani et al. 2013b].

6. Avaliação de Desempenho

Esta seção traz algumas avaliações sobre o *throughput* e a latência apresentada pelo BFT-SMART, além de compará-lo com outros sistemas similares.

Configuração. Os experimentos foram executados com três (CFT) e quatro (BFT) réplicas hospedadas em diferentes máquinas e até 1600 clientes foram distribuídos uniformemente em outras quatro máquinas. As máquinas são servidores Dell PowerEdge R410, configurados com JRE 1.7.0_21 e o Ubuntu Linux 10.04. Cada máquina possui 32GB de memória e dois processadores *quad-core* 2.27 GHz Intel Xeon E5520 com *hyper-threading*, i.e., suportando 16 *threads* em *hardware*. Todas máquinas se comunicam através de uma rede Ethernet *gigabit*.

Micro-benchmarks. Usamos um *micro-benchmark* comumente utilizado para avaliar sistemas SMR, que consiste na implementação de um serviço “vazio” com o BFT-SMART para calcular o *throughput* no lado dos servidores e a latência no lado dos clientes. Os resultados do *throughput* foram obtidos na réplica líder e a latência foi calculada em um dos clientes (sempre o mesmo). O desvio padrão sempre foi menor do que 3%.

Resultados. A Figura 2 apresenta os resultados para o BFT-SMART configurado tanto como CFT quanto BFT, considerando diferentes tamanhos para requisições/respostas: 0/0, 100/100, 1024/1024 e 4096/4096 *bytes*. Nesta figura podemos perceber que o BFT-SMART configurado como CFT possui um desempenho melhor do que quando configurado como BFT. Isso pode ser explicado pelo menor número de mensagens utilizadas na configuração CFT, o que resulta em menos trabalho para executar cada requisição.

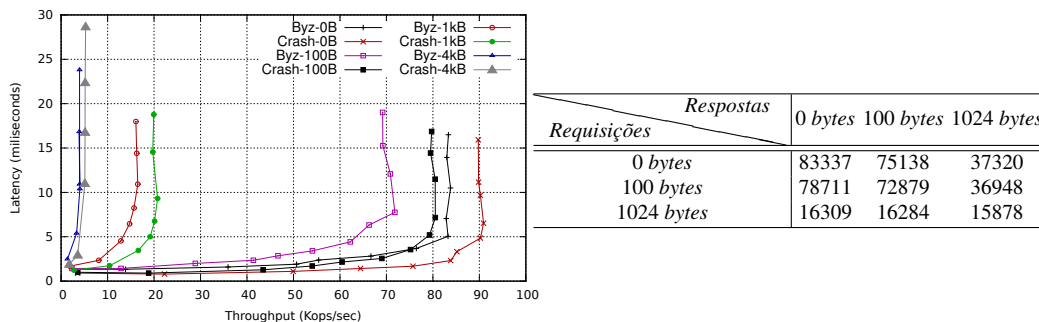


Figura 2. Latência vs. *Throughput* para $f = 1$ (esquerda) e *throughput* (op./seg.) para diferentes tamanhos de requisições/respostas e $f = 1$ (direita).

Estes resultados são completados com a tabela da direita da Figura 2, que mostra como as diferentes combinações de tamanhos de requisições e respostas afetam o *throughput*. Estes resultados foram obtidos com 1600 clientes acessando o sistema configurado como BFT. Podemos perceber que o tamanho da requisição tem uma maior influência no *throughput* do que o tamanho da resposta. Isso pode ser explicado pelo aumento no tamanho do lote de requisições a ser proposto no consenso.

Comparação com outros. Comparamos o BFT-SMART (BFT) com dois sistemas que implementam BFT SMR: o PBFT [Castro and Liskov 2002] e o UpRight [Clement et al. 2009]. Já o BFT-SMART (CFT) foi comparado com o JPaxos [Santos and Schiper 2013], que implementa um CFT SMR. O *benchmark* 0/0 foi utilizado nestes experimentos. A Tabela 1 apresenta o *throughput* máximo obtido para estes sistemas e o número de clientes requeridos para isso. Também apresentamos valores para o mesmo número de clientes, onde utilizamos 200 clientes (*Throughput* 200), que foi o número máximo suportado pelo PBFT. Os resultados mostram que o BFT-SMART apresentou um *throughput* maior do que os outros sistemas em nosso ambiente.

<i>Sistema</i>	<i>Throughput</i>	<i>Clientes</i>	<i>Throughput 200</i>
BFT-SMART (BFT)	83801	1000	66665
PBFT	78765	100	65603
UpRight	5160	600	3355
BFT-SMART (CFT)	90909	600	83834
JPaxos	62847	800	45407

Tabela 1. *Throughput* máximo para o benchmark 0/0 e $f = 1$.

7. Descrição da Demonstração Planejada

O potencial do BFT-SMART será explorado através da execução de um serviço de armazenamento tipo chave-valor replicado (descrito em [Bessani et al. 2013b]). Este serviço implementa a interface *Map* da API do Java e usa o BFT-SMART para replicação dos dados armazenados. As replicas podem ou não executar em máquinas diferentes, sendo que a demonstração deve abordar a ocorrência de falhas e de reconfigurações, visando a exploração de todos os mecanismos fornecidos pelo BFT-SMART.

8. Conclusões

Este artigo apresentou o BFT-SMART, uma ferramenta robusta e eficiente para implementação de BFT SMR. Os experimentos apresentados mostram que a implementação atual do BFT-SMART fornece um *throughput* maior do que outras propostas similares. O sistema aqui descrito é disponível como *software* de código aberto na página do projeto e atualmente existem vários grupos de pesquisa usando e modificando esta ferramenta de acordo com suas necessidades. Finalmente, uma descrição mais detalhada do BFT-SMART pode ser encontrada em [BFT-SMART, Bessani et al. 2013b].

Referências

- Amir, Y., Coan, B., Kirsch, J., and Lane, J. (2011). Prime: Byzantine replication under attack. *IEEE Transactions on Dependable and Secure Computing*, 8(4):564–577.
- Bessani, A., Santos, M., Felix, J., Neves, N., and Correia, M. (2013a). On the efficiency of durable state machine replication. In *Proc. of USENIX ATC 2013*.
- Bessani, A., Sousa, J., and Alchieri, E. (2013b). State machine replication for the masses with bft-smart. DI-FCUL-TR-2013-07, Department of Computer Science, University of Lisbon.
- BFT-SMART. High-performance byzantine fault-tolerant state machine replication. <http://code.google.com/p/bft-smart/>.
- Castro, M. and Liskov, B. (2002). Practical Byzantine fault-tolerance and proactive recovery. *ACM Transactions on Computer Systems*, 20(4):398–461.
- Chandra, T., Griesemer, R., and Redstone, J. (2007). Paxos made live - an engineering perspective (2006 invited talk). In *Proc. of the PODC'07*.
- Clement, A., Kapritsos, M., Lee, S., Wang, Y., Alvisi, L., Dahlin, M., and Riché, T. (2009). UpRight cluster services. In *Proc. of the ACM SOSP'09*.
- Guerraoui, R., Knežević, N., Quéma, V., and Vukolić, M. (2010). The next 700 BFT protocols. In *Proc. of ACM EuroSys'10*.
- Lamport, L., Malkhi, D., and Zhou, L. (2010). Reconfiguring a state machine. *SIGACT News*, 41(1).
- Santos, N. and Schiper, A. (2013). Achieving high-throughput State Machine Replication in multi-core systems. In *Proc. of the IEEE ICDSC'13*.
- Sousa, J. and Bessani, A. (2012). From Byzantine consensus to BFT state machine replication: A latency-optimal transformation. In *Proc. of EDCC'12*.
- Veronese, G., Correia, M., Bessani, A., Lung, L., and Verissimo, P. (2013). Efficient Byzantine fault tolerance. *IEEE Transactions on Computers*, 62(1).